

3. Les quatre piliers de la POO

3.1. Encapsulation

Définition simple

L'**encapsulation** consiste à **protéger les données d'un objet** en contrôlant la façon dont on y accède.

Au lieu de modifier directement les attributs, on passe par des **méthodes**.

Idée à retenir

Un objet garde ses données “à l'intérieur” et expose seulement ce qui est nécessaire.

Comme une **capsule** : le contenu est protégé.

Exemple simple

```
class CompteBancaire {  
    private double solde;  
  
    public void deposer(double montant) {  
        if (montant > 0) {  
            solde += montant;  
        }  
    }  
  
    public double getSolde() {  
        return solde;  
    }  
}
```

}

}

Ici :

- `solde` est privé : on ne peut pas le modifier directement.
- `deposer()` contrôle les dépôts.
- `getSolde()` permet seulement de consulter le solde.

Pourquoi c'est utile ?

L'encapsulation permet de :

- protéger les données ;
- éviter les modifications incorrectes ;
- mieux organiser le code ;
- rendre l'objet plus fiable.

Phrase à mémoriser

Encapsulation = protéger les données et contrôler leur accès.

3.2. Abstraction

Définition simple

L'**abstraction** consiste à **montrer uniquement l'essentiel** et à **cacher les détails compliqués**.

On se concentre sur **ce que l'objet fait**, pas sur **comment il le fait**.

Idée à retenir

L'abstraction simplifie l'utilisation d'un objet.

Comme une **télécommande** : tu appuies sur un bouton, sans savoir comment le circuit fonctionne.

Exemple simple

```
abstract class Animal {  
    abstract void faireDuBruit();  
}  
  
class Chien extends Animal {  
    void faireDuBruit() {  
        System.out.println("Wouf");  
    }  
}
```

Ici :

- `Animal` définit une idée générale ;
- `faireDuBruit()` indique ce qu'un animal doit faire ;
- `Chien` précise comment il le fait.

Pourquoi c'est utile ?

L'abstraction permet de :

- simplifier le code ;
- cacher les détails inutiles ;
- créer des modèles généraux ;
- rendre le programme plus facile à comprendre.

Phrase à mémoriser

Abstraction = montrer l'essentiel, cacher les détails.

3.3. Héritage

Définition simple

L'**héritage** permet à une classe de **récupérer les attributs et méthodes** d'une autre classe.

Une classe peut donc être construite à partir d'une classe plus générale.

Idée à retenir

L'héritage évite de répéter le même code.

Comme dans une famille : un enfant peut hériter de certaines caractéristiques de ses parents.

Exemple simple

```
class Animal {  
    void manger() {  
        System.out.println("L'animal mange");  
    }  
}
```

```
class Chien extends Animal {  
    void aboyer() {  
        System.out.println("Wouf");  
    }  
}
```

Ici :

- Animal est la classe parent ;
- Chien est la classe enfant ;
- Chien hérite de la méthode manger() .

Pourquoi c'est utile ?

L'héritage permet de :

- réutiliser du code ;
- organiser les classes ;
- créer des relations logiques ;
- spécialiser une classe générale.

Phrase à mémoriser

Héritage = une classe enfant réutilise le code d'une classe parent.

3.4. Polymorphisme

Définition simple

Le **polymorphisme** permet à une même méthode d'avoir **plusieurs comportements** selon l'objet qui l'utilise.

Le même appel peut produire des résultats différents.

Idée à retenir

Polymorphisme signifie : **plusieurs formes**.

Comme le bouton “**Lire**” : il peut lancer une musique, une vidéo ou un livre audio selon le contenu.

Exemple simple

```
class Animal {  
    void faireDuBruit() {  
        System.out.println("Bruit d'animal");  
    }  
}
```

```
class Chien extends Animal {  
    void faireDuBruit() {  
        System.out.println("Wouf");  
    }  
}
```

```
class Chat extends Animal {  
    void faireDuBruit() {  
        System.out.println("Miaou");  
    }  
}
```

Ici :

- `faireDuBruit()` existe dans plusieurs classes ;
- chaque classe donne sa propre version ;
- le comportement dépend de l'objet réel.

Pourquoi c'est utile ?

Le polymorphisme permet de :

- écrire du code plus flexible ;
- utiliser le même nom de méthode ;
- adapter le comportement selon l'objet ;
- éviter beaucoup de conditions inutiles.

Phrase à mémoriser

Polymorphisme = même action, comportements différents selon l'objet.

3.5. Relations entre les quatre piliers

Idée simple

Les quatre piliers de la POO travaillent ensemble pour créer des objets **clairs, protégés, réutilisables et flexibles**.

Les rôles essentiels

- **Encapsulation** : protège les données.
- **Abstraction** : montre seulement l'essentiel.
- **Héritage** : permet de réutiliser du code.
- **Polymorphisme** : permet plusieurs comportements avec une même action.

Exemple mental simple

Imagine une application avec des animaux :

- `Animal` définit l'idée générale : **abstraction**.
- Ses données sont protégées : **encapsulation**.
- `Chien` et `Chat` héritent de `Animal` : **héritage**.
- Chaque animal fait un bruit différent : **polymorphisme**.

Phrase à mémoriser

Les quatre piliers permettent de construire des objets bien organisés, sûrs et adaptables.

3.6. Erreurs fréquentes dans la compréhension des piliers

Erreur 1 : confondre encapsulation et abstraction

- **Encapsulation** : protège les données.
- **Abstraction** : cache les détails inutiles.

À retenir :

Encapsulation = protection.

Abstraction = simplification.

Erreur 2 : croire que l'héritage sert toujours

L'**héritage** est utile pour réutiliser du code, mais il ne faut pas l'utiliser partout.

Exemple :

```
class Voiture extends Moteur {  
  
}
```

Ce n'est pas logique : une voiture **a un moteur**, mais n'est pas un moteur.

Erreur 3 : confondre héritage et polymorphisme

- **Héritage** : une classe enfant récupère le code d'une classe parent.
- **Polymorphisme** : une même méthode peut agir différemment selon l'objet.

Phrase à mémoriser

Protection, simplification, réutilisation, adaptation : voilà les quatre piliers sans confusion.

3.7. Exemples simples des quatre piliers

Exemple global : un animal

```
abstract class Animal {  
  
    private String nom;  
  
  
    public Animal(String nom) {  
  
        this.nom = nom;  
  
    }  
}
```



```
}
```

```
public String getNom() {  
    return nom;  
}
```

```
abstract void faireDuBruit();  
}
```

```
class Chien extends Animal {  
    public Chien(String nom) {  
        super(nom);  
    }  
  
    void faireDuBruit() {  
        System.out.println("Wouf");  
    }  
}
```

Où sont les quatre piliers ?

- **Encapsulation** : `nom` est privé et accessible avec `getNom()`.
- **Abstraction** : `Animal` représente une idée générale.
- **Héritage** : `Chien` hérite de `Animal`.
- **Polymorphisme** : `faireDuBruit()` peut changer selon l'animal.

Idée à retenir

Un même exemple peut montrer les quatre piliers :

protéger, simplifier, réutiliser, adapter.

Phrase à mémoriser

Les quatre piliers rendent les objets plus clairs, sûrs et flexibles.

3.8. Importance des piliers dans le code professionnel

Idée simple

Dans un projet professionnel, les quatre piliers de la POO permettent d'écrire un code **plus propre, plus fiable et plus facile à faire évoluer.**

Pourquoi c'est important ?

- **Encapsulation** : évite les modifications dangereuses.
- **Abstraction** : simplifie l'utilisation du code.
- **Héritage** : réduit la répétition quand il est bien utilisé.
- **Polymorphisme** : rend le code plus flexible.

Exemple concret

Dans une application de paiement :

```
interface Paiement {  
    void payer();  
}
```

```
class PaiementCarte implements Paiement {  
    public void payer() {
```

```
        System.out.println("Païement par  
carte");  
    }  
}
```

```
class PaiementMobile implements Paiement {  
    public void payer() {  
        System.out.println("Païement mobile");  
    }  
}
```

Le programme utilise `payer()` sans connaître tous les détails internes.

Phrase à mémoriser

En code professionnel, les piliers de la POO rendent le logiciel plus maintenable, sécurisé et adaptable.